PHY 3902 Experiment 2 Lab Report

Author: Patrick Geraghty
Affiliation: University of Ottawa
Date: September 28, 2025

Abstract

This experiment investigates the properties of light-emitting diodes (LEDs) by analyzing their current-voltage (I-V) characteristics. The saturation current and ideality factor for each diode are determined from their I-V curves. Threshold voltages are measured and correlated with the emission wavelengths of the LEDs. By plotting threshold voltage against inverse wavelength and performing linear regression, an experimental value for Planck's constant is obtained. The results demonstrate the relationship between electrical energy and light emission, and provide a hands-on determination of a fundamental physical constant.

1 Introduction

The behavior of light-emitting diodes (LEDs) provides a practical means to explore the relationship between electrical energy and light emission. LEDs emit light at specific wavelengths, which are related to the energy required to activate the diode. In this experiment, the I-V characteristics of four different LEDs are measured and analyzed to extract their saturation currents and ideality factors, providing insight into basic diode behavior. The threshold voltage required to activate each LED is also determined. By plotting the threshold voltage against the inverse wavelength of the LEDs and performing a linear fit, an experimental value for Planck's constant is extracted. This approach provides experience with data analysis, uncertainty, and the determination of physical constants.

2 Methods

The experiment consisted of two main parts: characterization of LED I-V curves and determination of Planck's constant using threshold voltages.

2.1 Apparatus and Data Collection

A custom voltage divider circuit was built to measure the current-voltage (I-V) characteristics of four LEDs (blue, green, yellow, and red). Data acquisition was automated using an Arduino microcontroller and a digital-to-analog converter (DAC), with custom Arduino sketches (see Supplementary Code) controlling voltage sweeps and recording current measurements.

LED Wavelengths

The nominal emission wavelengths and their uncertainties for the LEDs used in this experiment were:

- **Blue**: 470 nm (range: 460–475 nm)
- **Green**: 520–525 nm
- **Red**: 623 nm (range: 620–630 nm)
- **Yellow**: 589 nm (range: 585–595 nm)

These wavelength ranges were used to estimate uncertainties in the inverse wavelength for the analysis of threshold voltage and Planck's constant.

2.2 I-V Curve Analysis

The I-V data for each LED was analyzed using the code listing *LED I-V Curve Fitting and Parameter Extraction* (Listing 1) in section A. The LED current as a function of voltage was modeled by the diode equation:

$$I(V) = I_s \left(e^{V/(nV_T)} - 1\right) \tag{1}$$

where I_s is the saturation current, n is the ideality factor, and V_T is the thermal voltage at room temperature. The thermal voltage is defined as $V_T = \frac{kT}{q}$, where k is Boltzmann's constant, T is the absolute temperature, and q is the elementary charge. Nonlinear curve fitting (using SciPy's `curve_fit`) was performed to extract I_s and n for each LED. Measurement uncertainties were propagated and cleaned to ensure robust fits.

2.3 Threshold Voltage and Planck's Constant

Threshold voltages were determined for each LED by identifying the voltage at which the current exceeded 0.01 mA and remained above this value. The code listing *LED Threshold Analysis and Planck's Constant Estimation* (Listing 2) in section A automated this process and correlated threshold voltages with the emission wavelengths of the LEDs (as specified by in-lab equipment). The inverse wavelength ($1/\lambda$) was plotted against threshold voltage, and linear regression was performed. The slope of the fit was used to estimate Planck's constant via the relation:

$$h = \text{slope} \cdot e/c \tag{2}$$

where e is the elementary charge and c is the speed of light.

2.4 Software and Analysis

All data analysis was performed in Python 3 using custom scripts adapted from provided reference scripts and literature. Figures and tables were generated using Matplotlib. Relevant code is included in the supplementary section at the end of this report.

3 Results and Discussion

3.1 I-V Curve Analysis

The current-voltage (I-V) characteristics of four LEDs (blue, green, yellow, and red) was measured and analyzed. The experimental data was fitted to the diode equation (1) to

LED Color	Saturation Current I_s (A)	Ideality Factor n
Blue	9.37×10^{-10}	7.10
Green	5.86×10^{-9}	7.17
Red	1.98×10^{-8}	5.56
Yellow	6.50×10^{-11}	3.93

Table 1: Fitted saturation current and ideality factor for each LED.

extract the saturation current (I_s) and ideality factor (n) for each LED. The results are summarized in Table 1.

Figure 1 shows the measured I-V curves and the fitted diode model for all LEDs. The fits are generally good, though the ideality factors are higher than typical values for silicon diodes.

A closer inspection of the individual and group I-V plots (see Figures 1 and 2) reveals a key source of error in the curve fitting process. While the fitted diode equation tracks the experimental data well at both low and high voltages, it diverges noticeably in the "elbow" region, where the current begins to increase rapidly. In this region, the actual LED data exhibits a much sharper transition than the fitted curve, which rises more gradually. This difference means the real LED behaves more like an ideal diode, with a sudden turn-on, whereas the model underestimates the rate of change. As a result, the extracted ideality factor is systematically higher than expected. The ideality factor is highly sensitive to the steepness of the curve at the elbow, so any inability of the model to capture this sharpness leads to significant error. This limitation is inherent to the diode equation fit and is not simply a result of measurement uncertainty or data cleaning. Addressing this modeling mismatch would require either a more sophisticated physical model or a targeted fitting approach that better accounts for the rapid turn-on behavior of real LEDs.

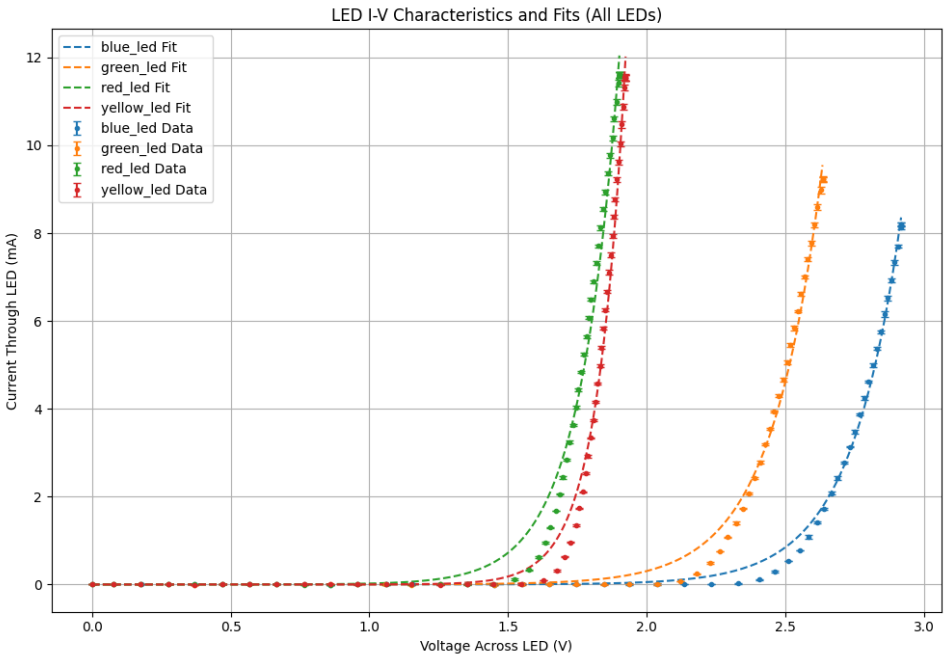


Figure 1: Measured I-V curves and fitted diode model for blue, green, yellow, and red LEDs. Each set of points represents experimental data for a specific LED, with error bars indicating measurement uncertainty in current. The dashed lines show the best-fit diode equation for each LED, from which the saturation current and ideality factor were extracted. The plot highlights the exponential increase in current with voltage and the differences in turn-on behavior between LED colors. The quality of fit and the spread of data points illustrate both the strengths and limitations of the diode model for real LEDs.

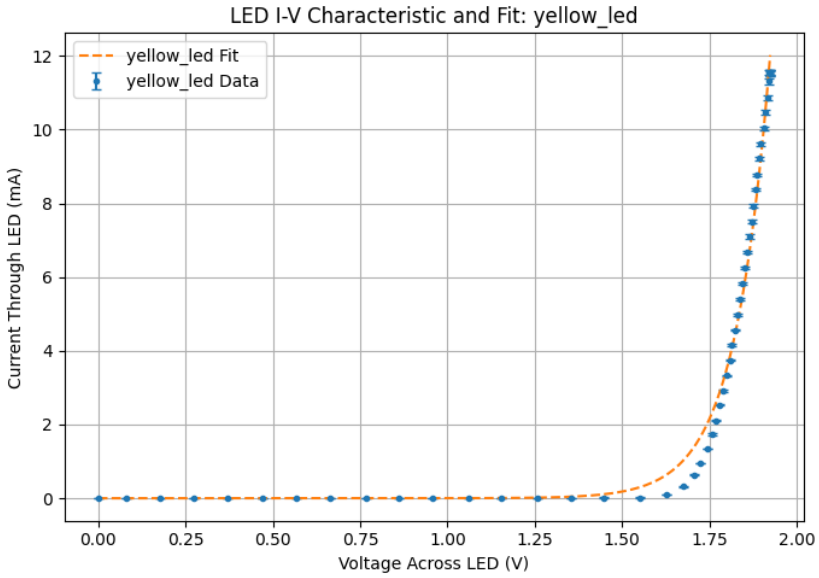


Figure 2: Measured I-V curve and fitted diode model for the yellow LED. This plot shows the experimental data points with error bars and the best-fit diode equation for the yellow LED only, illustrating its individual turn-on behavior and fit quality.

The I-V curve analysis provides insight into the electrical behavior of the LEDs, but several limitations and sources of error should be considered. The fitted ideality factors

(ranging from 3.93 to 7.17) are significantly higher than typical values for standard diodes (usually 1–2), indicating non-ideal behavior. This may arise from measurement errors, contact resistance, or deviations from the ideal diode model in real LEDs. The cleaning of current error values by replacing non-positive or NaN entries with a small constant (1e-9 A) may not accurately reflect the true measurement uncertainty, potentially biasing the analysis.

Other sources of error include temperature variations during data acquisition, and noise in the current measurements. The arbitrary choice of a current cutoff (0.01 mA) for determining threshold voltage may also introduce systematic bias. To improve the reliability of the I-V analysis, future work should consider more robust error estimation, and careful control of experimental conditions. Overall, while the I-V analysis yields reasonable fits and trends, the extracted parameters and their uncertainties should be interpreted with caution due to these limitations.

A paper by Guo Wei-Ling, et al. on the electrical characteristics of high-voltage LEDs [1], provided valuable insight into the electrical characteristics of LEDs, particularly regarding the ideality factor. Although the paper focuses on high-voltage GaN-based LEDs, which differ from the standard LEDs used in our experiment, it discusses similar observations of high ideality factors and possible contributing factors. While not all findings are directly applicable to our setup, it helped inform our understanding of non-ideal diode behavior and the challenges in interpreting experimental results.

3.2 Threshold Voltage and Planck's Constant

Threshold voltages for each LED were determined and plotted against the inverse of their emission wavelength. Figure 3 shows the resulting graph and linear fit. The slope of the fit was used to estimate Planck's constant using equation (2).

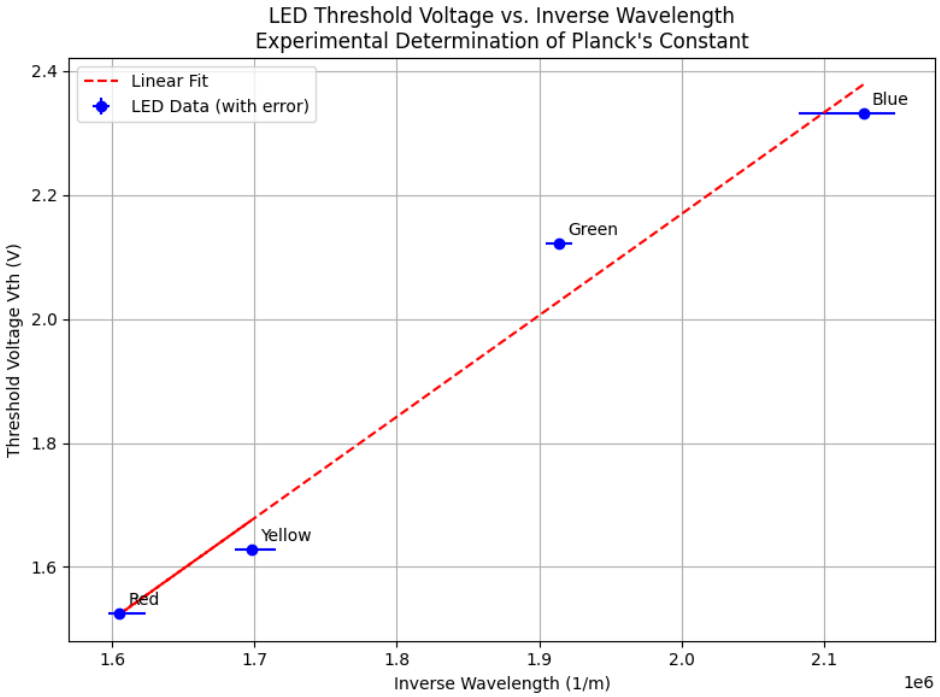


Figure 3: Threshold voltage versus inverse wavelength for the four LEDs. Each data point corresponds to a different LED, with horizontal error bars representing uncertainty in the inverse wavelength (based on manufacturer specifications) and vertical error bars for threshold voltage (based on measurement repeatability). The solid line shows the linear regression fit, whose slope is used to estimate Planck's constant. The plot demonstrates the expected linear relationship, but also reveals the impact of measurement and systematic uncertainties, as seen in the spread and size of the error bars.

The experimentally determined value of Planck's constant was $h = 8.742 \times 10^{-34}$ J s, compared to the accepted literature value of $h = 6.626 \times 10^{-34}$ J s. The percent error was 31.94%. The R^2 value of the fit was 0.965, indicating a strong linear relationship between threshold voltage and inverse wavelength. The uncertainty in the slope was $\pm 2.22 \times 10^{-7}$ V m.

The threshold voltage analysis and subsequent determination of Planck's constant demonstrate the relationship between LED threshold voltage and emission wavelength. However, a closer inspection of the plot in Figure 3 reveals a notable issue: the vertical error bars (uncertainty in threshold voltage) are nearly invisible, while the horizontal error bars (uncertainty in inverse wavelength) are relatively large. Despite the small vertical uncertainties, the experimentally determined value of Planck's constant deviates by 31% from the accepted value. This discrepancy is not reflected by the error bars, indicating that the uncertainties in threshold voltage are likely underestimated or that significant systematic errors are present.

Possible sources of error include:

- Underestimation of threshold voltage uncertainty due to limitations in the measurement method or data analysis (e.g., the current cutoff criterion).
- Systematic errors in the determination of threshold voltage, such as contact resistance, temperature effects, or non-ideal diode behavior.

- Inaccuracies in the specified LED wavelengths, which contribute to the large horizontal error bars but do not fully explain the deviation in Planck's constant.
- The assumption that the threshold voltage directly corresponds to photon energy may neglect other physical effects in the LED junction.

The strong linear correlation ($R^2 = 0.965$) suggests the method is valid in principle, but the error analysis does not realistically capture the true uncertainty in the experimental value of h. To improve reliability, future work should focus on more robust estimation of threshold voltage uncertainty, careful consideration of systematic effects, and possibly alternative methods for threshold determination. Overall, while the experiment successfully demonstrates the relationship between LED threshold voltage and emission wavelength, the reported uncertainty does not account for the observed deviation, and the result should be interpreted with caution.

In comparison to previous work, such as the experiment described by Calvo-Mola et al. [2], our goal was to demonstrate the same fundamental concepts using a similar approach and comparable equipment. However, despite these similarities, our results were less accurate and had larger uncertainties. Their report details a more careful measurement process, including precise calibration, accurate determination of LED wavelengths, and refined methods for extracting threshold voltages. As a result, their experimental value for Planck's constant was much closer to the accepted value. In our experiment, factors such as measurement noise, less rigorous calibration, and a simpler approach to threshold voltage determination contributed to the larger deviation and underestimated uncertainties. This experiment was used as a guide for methodology and analysis, and our results highlight the importance of attention to detail and thorough error analysis for achieving high accuracy in physical constant determination.

4 Conclusion

This experiment provided a comprehensive investigation into the electrical properties of LEDs. By analyzing the I-V characteristics, we extracted saturation currents and ideality factors, revealing significant non-ideal behavior and highlighting the importance of careful error analysis and experimental design. The determination of Planck's constant from threshold voltage and wavelength data demonstrated the relationship between LED threshold voltage and emission wavelength, but also exposed limitations in uncertainty estimation and the impact of systematic errors. While the strong linear correlation supports the underlying physical relationship, the large deviation from the accepted value of Planck's constant and the underestimated uncertainties indicate that further refinement of methods is needed. Future work should focus on improving measurement accuracy, error propagation, and accounting for systematic effects. Overall, the experiment successfully illustrated basic concepts in diode behavior and the challenges of precise experimental determination of fundamental constants.

References

[1] Guo W-L, Yan W-W, Zhu Y-X, et al; "Analysis on electrical characteristics of high-voltage gan-based light-emitting diodes"; Chinese Physics B 21:127201; doi: 10.1088/1674-1056/21/12/127201 (2012)

[2] C. Calvo-Mola, S. López-Pérez, E. Garcıa-Alfonso, J. Cerutti-Torres; “Determination Of The Planck Constant Through The Use Of LEDS” (2019)

A Supplementary: Code

A.1 Python Code

```python
import matplotlib.pyplot as plt
import numpy as np
import os
import pandas as pd
from scipy.optimize import curve_fit
from scipy.constants import k, e
from typing import Dict, Any, List, Tuple

OUT_DIR = '../out'
os.makedirs(OUT_DIR, exist_ok=True)

def led_iv_curve(V: np.ndarray, I_s: float, n: float) -> np.ndarray:
    """
    LED I-V characteristic equation.

    Args:
        V (np.ndarray): Voltage across the LED (V)
        I_s (float): Saturation current (A)
        n (float): Ideality factor (dimensionless)

    Returns:
        np.ndarray: Current through the LED (A)
    """
    V_T = k * 298.15 / e  # Thermal voltage at room temp (T=298.15K)
    return I_s * (np.exp(V / (n * V_T)) - 1)

def load_led_data(csv_file: str) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Load LED I-V data from a CSV file.

    Args:
        csv_file (str): Path to the CSV file.

    Returns:
        Tuple[np.ndarray, np.ndarray, np.ndarray]: Voltage, current, and current error arrays.
    """
    df = pd.read_csv(csv_file)
    V = df["A1-A2"].values
    I = df["current (mA)"].values * 1e-3  # Convert mA to A
    dI = df["d(current (mA))"].values * 1e-3  # Convert mA to A
    return V, I, dI

def fit_led_iv(V: np.ndarray, I: np.ndarray) -> Tuple[float, float, bool]:
    """
    Fit LED I-V data to the diode equation.

```

```
    Args:
        V (np.ndarray): Voltage array.
        I (np.ndarray): Current array.

    Returns:
        Tuple[float, float, bool]: Fitted I_s, n, and success flag.
    """
    try:
        popt, _ = curve_fit(
            led_iv_curve, V, I, bounds=(0, [1e-6, 10]), maxfev=10000
        )
        I_s_fit, n_fit = popt
        return I_s_fit, n_fit, True
    except Exception:
        return np.nan, np.nan, False

def clean_current_error(dI: np.ndarray) -> np.ndarray:
    """
    Clean current error array by replacing non-positive or NaN values
       with a small value.

    Args:
        dI (np.ndarray): Current error array.

    Returns:
        np.ndarray: Cleaned current error array.
    """
    return np.where((dI > 0) & np.isfinite(dI), dI, 1e-9)

def process_led_file(csv_file: str) -> Dict[str, Any]:
    """
    Process a single LED CSV file: load data, fit, and clean errors.

    Args:
        csv_file (str): Path to the CSV file.

    Returns:
        Dict[str, Any]: Results including voltage, current, error, fit
           params, and success flag.
    """
    V, I, dI = load_led_data(csv_file)
    I_s_fit, n_fit, fit_success = fit_led_iv(V, I)
    dI_clean = clean_current_error(dI)
    return {
        "name": os.path.basename(csv_file),
        "V": V,
        "I": I,
        "dI": dI_clean,
        "fit_success": fit_success,
        "I_s_fit": I_s_fit,
        "n_fit": n_fit
    }

def plot_led_results(led_results: List[Dict[str, Any]], title: str = "
   LED I-V Characteristics and Fits (All LEDs)") -> None:
    """
    Plot measured and fitted I-V data for multiple LEDs.

```

```
    Args:
        led_results (List[Dict[str, Any]]): List of LED result dicts.
        title (str): Plot title.
    """
    plt.figure(figsize=(10, 7))
    colors = plt.cm.tab10.colors
    for idx, result in enumerate(led_results):
        color = colors[idx % len(colors)]
        label = result["name"].replace(".csv", "").replace("2d_", "")
        plt.errorbar(result["V"], result["I"] * 1e3, yerr=result["dI"]
            * 1e3, fmt='o', label=f"{label} Data", color=color, capsize
            =3, markersize=3)
        if result["fit_success"]:
            V_fit = np.linspace(result["V"].min(), result["V"].max(),
                200)
            I_fit = led_iv_curve(V_fit, result["I_s_fit"], result["
                n_fit"])
            plt.plot(V_fit, I_fit * 1e3, label=f"{label} Fit", color=
                color, linestyle='--')
    plt.xlabel("Voltage Across LED (V)")
    plt.ylabel("Current Through LED (mA)")
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    out_path = os.path.join(OUT_DIR, '2d_led_fit_plot.png')
    plt.savefig(out_path)
    plt.close()

def plot_single_led_result(result: Dict[str, Any], out_dir: str =
    OUT_DIR) -> None:
    """
    Plot measured and fitted I-V data for a single LED and save to a
        file.

    Args:
        result (Dict[str, Any]): LED result dict.
        out_dir (str): Output directory for plots.
    """
    plt.figure(figsize=(7, 5))
    label = result["name"].replace(".csv", "").replace("2d_", "")
    plt.errorbar(result["V"], result["I"] * 1e3, yerr=result["dI"] * 1
        e3, fmt='o', label=f"{label} Data", capsize=3, markersize=3)
    if result["fit_success"]:
        V_fit = np.linspace(result["V"].min(), result["V"].max(), 200)
        I_fit = led_iv_curve(V_fit, result["I_s_fit"], result["n_fit"])
        plt.plot(V_fit, I_fit * 1e3, label=f"{label} Fit", linestyle='
            --')
    plt.xlabel("Voltage Across LED (V)")
    plt.ylabel("Current Through LED (mA)")
    plt.title(f"LED I-V Characteristic and Fit: {label}")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    out_path = os.path.join(out_dir, f"{label}_fit.png")
    plt.savefig(out_path)
    plt.close()

```

```python
def main(data_dir: str = "../data/2d") -> None:
    """
    Main execution function: processes all LED CSV files and plots
       results.

    Args:
        data_dir (str): Directory containing LED CSV files.
    """
    led_files = [fname for fname in os.listdir(data_dir) if fname.
       endswith(".csv")]
    led_results = []
    for fname in led_files:
        csv_path = os.path.join(data_dir, fname)
        result = process_led_file(csv_path)
        led_results.append(result)
        if result["fit_success"]:
            print(f"{result['name']}:")
            print(f"  Fitted saturation current I_s: {result['I_s_fit
               ']:.2e} A")
            print(f"  Fitted ideality factor n: {result['n_fit']:.2f}\n
               ")
        else:
            print(f"{result['name']}: Fit failed\n")
        plot_single_led_result(result, OUT_DIR)  # Save individual plot
            for each LED
    plot_led_results(led_results)

if __name__ == "__main__":
    main()
```

Listing 1: LED I-V Curve Fitting and Parameter Extraction

```python
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.constants import c, e
from scipy.stats import linregress
from typing import Tuple, List, Dict

# --- Configuration ---
DATA_DIR = '../data/2d'
OUT_DIR = '../out'
# LED wavelength specs: (central, lower, upper)
LED_WAVELENGTHS_NM = {
    'blue': (470, 460, 475),
    'green': (522.5, 520, 525),
    'yellow': (589, 585, 595),
    'red': (623, 620, 630),
}
LED_COLOR_MAP = {
    '2d_blue_led.csv': 'blue',
    '2d_green_led.csv': 'green',
    '2d_yellow_led.csv': 'yellow',
    '2d_red_led.csv': 'red'
}
os.makedirs(OUT_DIR, exist_ok=True)

```

```python
# --- Data Extraction Functions ---
def get_threshold_voltage(csv_path: str) -> Tuple[float, float]:
    """
    Extract the threshold voltage and its error from a CSV file.
    The threshold voltage is defined as the first voltage where all subsequent currents > 0.01 mA.
    The error is taken from the d(A1-A2) column at the threshold index.
    Args:
        csv_path: Path to the CSV file.
    Returns:
        (threshold_voltage, threshold_voltage_error)
    """
    df = pd.read_csv(csv_path)
    voltage_col = 'A1-A2'
    current_col = 'current (mA)'
    voltage_error_col = 'd(A1-A2)'
    V = df[voltage_col].values
    I = df[current_col].values
    dV = df[voltage_error_col].values
    for i in range(len(I)):
        if np.all(I[i:] > 0.01):
            return V[i], dV[i]
    return np.nan, np.nan

# --- Gather LED Data ---
def gather_led_data(data_dir: str, color_map: Dict[str, str], wavelengths_nm: Dict[str, Tuple[float, float, float]]) -> Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray, List[str]]:
    """
    Gather threshold voltages, errors, wavelengths, and colors for all LED CSV files in the directory.
    Args:
        data_dir: Directory containing CSV files.
        color_map: Mapping from filename to LED color.
        wavelengths_nm: Mapping from color to wavelength (nm).
    Returns:
        vths: Array of threshold voltages.
        vth_errs: Array of threshold voltage errors.
        wavelengths_nm_arr: Array of wavelengths (nm).
        wl_err_low_arr: Array of lower wavelength errors (nm).
        wl_err_high_arr: Array of upper wavelength errors (nm).
        colors: List of LED colors.
    """
    vths, vth_errs, wavelengths_nm_arr, wl_err_low_arr, wl_err_high_arr, colors = [], [], [], [], [], []
    for fname in os.listdir(data_dir):
        if fname.endswith('.csv') and fname in color_map:
            color = color_map[fname]
            csv_path = os.path.join(data_dir, fname)
            vth, err = get_threshold_voltage(csv_path)
            central, low, high = wavelengths_nm[color]
            vths.append(vth)
            vth_errs.append(err)
            wavelengths_nm_arr.append(central)
            wl_err_low_arr.append(central - low)
            wl_err_high_arr.append(high - central)
            colors.append(color)
    return (np.array(vths), np.array(vth_errs), np.array(
```

```
        wavelengths_nm_arr), np.array(wl_err_low_arr), np.array(
        wl_err_high_arr), colors)

# --- Analysis Functions ---
def calculate_plancks_constant(slope: float) -> float:
    """
    Calculate Planck's constant from the slope of the fit.
    Args:
        slope: Slope from linear fit (V*m).
    Returns:
        Experimental Planck's constant (J*s).
    """
    return slope * e / c

# --- Plotting Function ---
def plot_led_data(inv_wavelengths: np.ndarray, inv_wavelengths_err_low:
    np.ndarray, inv_wavelengths_err_high: np.ndarray, vths: np.ndarray,
    vth_errs: np.ndarray, slope: float, intercept: float, colors: List[
   str],) -> None:
    """
    Plot threshold voltage vs inverse wavelength with error bars and
       linear fit.
    Args:
        inv_wavelengths: Array of inverse wavelengths (1/m).
        inv_wavelengths_err_low: Array of lower inverse wavelength
           errors.
        inv_wavelengths_err_high: Array of upper inverse wavelength
           errors.
        vths: Array of threshold voltages (V).
        vth_errs: Array of threshold voltage errors.
        slope: Slope from linear fit.
        intercept: Intercept from linear fit.
        colors: List of LED colors for annotation.
    """
    plt.figure(figsize=(8,6))
    # Asymmetric error bars for x (inverse wavelength)
    plt.errorbar(inv_wavelengths, vths, xerr=[inv_wavelengths_err_low,
       inv_wavelengths_err_high], yerr=vth_errs, fmt='o', color='blue',
        label='LED Data (with error)')
    plt.plot(inv_wavelengths, slope * inv_wavelengths + intercept, 'r--
       ', label='Linear Fit')
    for i, color in enumerate(colors):
        plt.annotate(color.capitalize(), (inv_wavelengths[i], vths[i]),
            textcoords="offset points", xytext=(5,5), ha='left')
    plt.xlabel('Inverse Wavelength (1/m)')
    plt.ylabel('Threshold Voltage Vth (V)')
    plt.title("LED Threshold Voltage vs. Inverse Wavelength\
       nExperimental Determination of Planck's Constant")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    out_path = os.path.join(OUT_DIR, 'led_threshold_vs_inv_wavelength.
       png')
    plt.savefig(out_path)
    plt.close()

# --- Main Execution ---
def main():
```

```
    """
    Main function to orchestrate LED data analysis and plotting.
    """
    vths, vth_errs, wavelengths_nm, wl_err_low, wl_err_high, colors =
        gather_led_data(DATA_DIR, LED_COLOR_MAP, LED_WAVELENGTHS_NM)
    wavelengths_m = wavelengths_nm * 1e-9
    wl_err_low_m = wl_err_low * 1e-9
    wl_err_high_m = wl_err_high * 1e-9
    inv_wavelengths = 1 / wavelengths_m
    # Asymmetric error propagation for inverse wavelength
    inv_wavelengths_err_low = wl_err_low_m / wavelengths_m**2
    inv_wavelengths_err_high = wl_err_high_m / wavelengths_m**2
    # Linear regression
    slope, intercept, r_value, p_value, std_err = linregress(
        inv_wavelengths, vths)
    h_exp = calculate_plancks_constant(slope)
    plot_led_data(inv_wavelengths, inv_wavelengths_err_low,
        inv_wavelengths_err_high, vths, vth_errs, slope, intercept,
        colors)
    print(f"Experimental␣Planck's␣constant:␣h␣=␣{h_exp:.3e}␣J*s")
    print(f"Literature␣value:␣h␣=␣6.626e-34␣J*s")
    print(f"Slope␣(hc/e):␣{slope:.3e}␣V*m␣+/-␣{std_err:.2e}")
    print(f"R^2␣of␣fit:␣{r_value**2:.3f}")
    print(f"Percent␣error␣in␣h:␣{abs((h_exp␣-␣6.626e-34)␣/␣6.626e-34)␣*
        ␣100:.2f}%")

if __name__ == "__main__":
    main()
```

Listing 2: LED Threshold Analysis and Planck's Constant Estimation

A.2 Arduino Code

```
#include <Wire.h>
#include <Adafruit_MCP4725.h>

// --- Structs ---

/**
 * @struct AnalogStats
 * @brief Holds the mean and standard deviation of analog readings.
 *
 * Used to store the average (mean) and standard deviation (stddev) of
    multiple analog readings.
 */
struct AnalogStats {
    float mean;   /**< The mean (average) value of the analog readings.
        */
    float stddev; /**< The standard deviation of the analog readings.
        */
};

/**
 * @brief Holds the result of a difference and its propagated error.
 */
struct DiffStats {
    float diff;
```

```
22      float error;
23  };
24
25  // --- Constants ---
26  const float RESISTOR = 218.3; // Ohms
27  const float RESISTOR_TOLERANCE = 0.005; // 0.5%
28  const float DIODE_RESISTANCE = 0;  // Ohms
29  const uint8_t DAC_ADDR = 0x62;
30  const int DAC_STEPS = 51;
31  const float DAC_VOLTAGE_STEP = 0.1;
32  const int DAC_RESOLUTION = 4095;
33  const int ANALOG_READ_DELAY = 50;
34
35  // --- Globals ---
36  bool hasRun = false;
37  Adafruit_MCP4725 dac;
38
39  // --- Utility Functions ---
40
41  /**
42   * @brief Calculates the difference and propagated error between two
        AnalogStats.
43   * @param a First AnalogStats.
44   * @param b Second AnalogStats.
45   * @return DiffStats struct with difference and propagated error.
46   */
47  DiffStats calcDiffStats(const AnalogStats& a, const AnalogStats& b) {
48      DiffStats result;
49      result.diff = a.mean - b.mean;
50      result.error = propagate_addition_stddev(a.mean, b.mean, a.stddev,
            b.stddev);
51      return result;
52  }
53
54  /**
55   * @brief Maps a float value from one range to another.
56   *
57   * @param value The input value to map.
58   * @param fromLow The lower bound of the input range.
59   * @param fromHigh The upper bound of the input range.
60   * @param toLow The lower bound of the output range.
61   * @param toHigh The upper bound of the output range.
62   * @return The mapped value in the output range.
63   */
64  float float_map(float value, float fromLow, float fromHigh, float toLow
        , float toHigh) {
65      return toLow + ((value - fromLow) * (toHigh - toLow)) / (fromHigh -
            fromLow);
66  }
67
68
69  /**
70   * @brief Propagates standard deviation for addition or subtraction of
        two values.
71   *
72   * Uses error propagation formula for addition/subtraction.
73   *
74   * @param a First value.
```

```
 * @param b Second value.
 * @param stddev_a Standard deviation of the first value.
 * @param stddev_b Standard deviation of the second value.
 * @return The propagated standard deviation.
 */
float propagate_addition_stddev(float a, float b, float stddev_a, float stddev_b) {
    return sqrt(pow(a, 2) * pow(stddev_a, 2) + pow(b, 2) * pow(stddev_b, 2));
}


/**
 * @brief Prints a value in CSV format to the Serial output.
 *
 * @tparam T The type of the value to print.
 * @param value The value to print.
 * @param eol If true, prints a newline after the value; otherwise, prints a comma.
 */
template <typename T>
void printSerialCSV(T value, bool eol = false, int precision = 4) {
    if (eol) {
        Serial.println(value, precision);
    } else {
        Serial.print(value, precision);
        Serial.print(",");
    }
}

/**
 * @brief Prints a full CSV row of the experiment data to Serial.
 * @param outputVoltage The DAC output voltage.
 * @param a0 AnalogStats for A0.
 * @param a1 AnalogStats for A1.
 * @param a2 AnalogStats for A2.
 * @param a0_a1 DiffStats for A0-A1.
 * @param a1_a2 DiffStats for A1-A2.
 * @param current The calculated current (mA).
 * @param currentError The propagated error for current (mA).
 */
void printDataCSVRow(float outputVoltage, const AnalogStats& a0, const AnalogStats& a1, const AnalogStats& a2,
                     const DiffStats& a0_a1, const DiffStats& a1_a2,
                        float current, float currentError) {
    Serial.print(String(outputVoltage) + ",");
    printSerialCSV(a0.mean);
    printSerialCSV(a0.stddev);
    printSerialCSV(a1.mean);
    printSerialCSV(a1.stddev);
    printSerialCSV(a2.mean);
    printSerialCSV(a2.stddev);
    printSerialCSV(a0_a1.diff);
    printSerialCSV(a0_a1.error);
    printSerialCSV(a1_a2.diff);
    printSerialCSV(a1_a2.error);
    printSerialCSV(current);
    printSerialCSV(currentError, true);
```

```
}

/**
 * @brief Reads the analog value from a pin and converts it to a voltage (0-5V).
 *
 * Takes multiple samples for stability and calculates mean and standard deviation.
 *
 * @param pin The analog pin to read from.
 * @param samples Number of samples to average (default: 10, max: 20).
 * @return AnalogStats struct containing mean and stddev of the voltage readings.
 */
AnalogStats readAnalogToVoltage(uint8_t pin, int samples = 10) {
    float values[20]; // Support up to 20 samples
    float total = 0;
    for (int i = 0; i < samples; i++) {
        values[i] = analogRead(pin);
        total += values[i];
        delay(5);
    }
    float mean = total / samples;
    float variance = 0;
    for (int i = 0; i < samples; i++) {
        variance += (values[i] - mean) * (values[i] - mean);
    }
    variance /= samples;
    float stddev = sqrt(variance);

    // Map to voltage
    AnalogStats stats;
    stats.mean = float_map(mean, 0, 1023, 0, 5.0);
    stats.stddev = float_map(stddev, 0, 1023, 0, 5.0);
    return stats;
}

// --- Analog Pin Helpers ---

/**
 * @brief Reads and returns voltage statistics from analog pin A0.
 * @return AnalogStats struct for A0.
 */
AnalogStats readA0() {
    return readAnalogToVoltage(A0);
}

/**
 * @brief Reads and returns voltage statistics from analog pin A1.
 * @return AnalogStats struct for A1.
 */
AnalogStats readA1() {
    return readAnalogToVoltage(A1);
}

/**
 * @brief Reads and returns voltage statistics from analog pin A2.
 * @return AnalogStats struct for A2.
```

```
 */
AnalogStats readA2() {
    return readAnalogToVoltage(A2);
}

// --- DAC Control ---

/**
 * @brief Sets the DAC output voltage by step index.
 *
 * Maps the step to the DAC's resolution and sets the output voltage.
 *
 * @param step The step index (0 to 51).
 */
void setDACVoltageByStep(int step) {
    int outputSignal = float_map(step, 0, 51, 0, 4095);
    dac.setVoltage(outputSignal, false);
}

// --- Data Acquisition ---

/**
 * @brief Acquires analog data for a given DAC step and prints results in CSV format.
 *
 * Sets the DAC output, reads analog pins, computes differences and currents, and prints all values.
 *
 * @param step The DAC step index.
 */
void acquireAndPrintData(int step) {
    float outputVoltage = step / 10.0;
    setDACVoltageByStep(step);
    delay(50);

    AnalogStats signalA0 = readA0();
    AnalogStats signalA1 = readA1();
    AnalogStats signalA2 = readA2();


    DiffStats a0_a1 = calcDiffStats(signalA0, signalA1);
    DiffStats a1_a2 = calcDiffStats(signalA1, signalA2);
    float current = (a0_a1.diff / RESISTOR) * 1000;
    // Error propagation for I = V/R: dI = I * sqrt( (dV/V)^2 + (dR/R)^2 )
    float dV = a0_a1.error;
    float dR = RESISTOR * RESISTOR_TOLERANCE;
    float currentError = current * sqrt( pow(dV / a0_a1.diff, 2 ) + pow (dR / RESISTOR, 2 ) );

    printDataCSVRow(outputVoltage, signalA0, signalA1, signalA2, a0_a1, a1_a2, current, currentError);

    delay(50);
}

// --- Header Printing ---

```

```
/**
 * @brief Prints the CSV header and experiment information to Serial.
 */
void printHeader() {
    for (int i = 0; i < 35; i++) Serial.print("-");
    Serial.println();
    Serial.println("DAC mounted to 0x62");
    Serial.println("Outputting voltage series.");
    Serial.println("out,A0,dA0,A1,dA1,A2,dA2,A0-A1,d(A0-A1),A1-A2,d(A1-A2),current (mA),d(current (mA))");
}

// --- Arduino Setup ---

/**
 * @brief Arduino setup function. Initializes Serial and DAC.
 */
void setup() {
    Serial.begin(9600);
    dac.begin(0x62);
    // for (int i = 0; i < 35; i++) {
    //     Serial.print("-");
    // }
    // Serial.println();
}

// --- Arduino Loop ---

/**
 * @brief Arduino main loop. Runs the experiment once and prints results.
 */
void loop() {
    if (!hasRun) {
        printHeader();
        for (int i = 0; i < 51; i++) {
            acquireAndPrintData(i);
        }
        Serial.println("Finished");
        hasRun = true;
    }
}
```

Listing 3: DAC LED Analyzer 2C

```
#include <Wire.h>
#include <Adafruit_MCP4725.h>

// - Variables
bool hasRun = false;
float outputVoltage;
float measuredInputSignal;
float measuredInputVoltage;
unsigned long analogReadPIN = A0;

Adafruit_MCP4725 dac;

void setup() {
```

```
  Serial.begin(9600);
  dac.begin(0x62);
  for (int i = 0; i < 35; i++) {
    Serial.print("-");
  }
  Serial.println();
  Serial.println("DAC␣mounted␣to␣0x62");
}

void loop() {
  if (not hasRun) {
    Serial.println("Outputting␣voltage␣series.");
    Serial.println("Ouput␣Voltage␣␣␣␣␣Measured␣Voltage");
    for (int i = 0; i < 51; i++) {
      int outputSignal = float_map(i, 0, 51, 0, 4095);
      dac.setVoltage(outputSignal, false);
      outputVoltage = i / 10.0;
      Serial.print(outputVoltage);

      delay(50);

      measuredInputSignal = analogRead(analogReadPIN);
      measuredInputVoltage = float_map(measuredInputSignal, 0, 1023, 0,
          50) / 10.0;
      measuredInputVoltage = measuredInputVoltage;

      int space_count = 18 - String(outputVoltage).length();
      spacer(space_count);

      Serial.println(measuredInputVoltage);
      delay(50);
    }
    Serial.println("Finished");
    hasRun = true;
  }
}

float float_map(float value, float fromLow, float fromHigh, float toLow
    , float toHigh) {
  return toLow + ((value - fromLow) * (toHigh - toLow)) / (fromHigh -
      fromLow);
}

void spacer(int num) {
  for (int i = 0; i < num; i++) {
    Serial.print("␣");
  }
}
```

Listing 4: DAC setup experiment 2B